

Selected Topics in Visual Recognition using Deep Learning

HW1 Report

111550028 黃柏翔

My Github Code link: https://github.com/Pakomu/CV_DeepLearning_2026/tree/main

I. Introduction

For this assignment, in addition to the standard ResNet baseline, my core methodology was inspired by the homework discussions. Since utilizing ResNeXt and ensemble learning were permitted, I decided to train three models and ensemble them for the final testing phase. Through a series of optimization steps to enhance training performance and boost the final evaluation score, I successfully completed the implementation.

II. Method

At first, in the data processing I did 3 methods on it: data augmentation [1], data normalization [2]. I apply the pytorch function of the data augmentation: RandAugment and TrivialAugmentWide to randomize data. And I resized 3 different size images for multiple training [3]. About the normalization, according to the internet, I apply the formula which lots of people use in CNN: mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225].

Then my loss function, optimizer, and scalar used nn.CrossEntropy [4], AdamW [5], LR_scheduler [6]. Otherwise, to decrease the calculation time, I use the method of mix precision training [7]. It can convert float32 to float16 and reduce the calculation time. However, I am not sure if it would affect the training performance.

Then I do the classic propagation, including compute the loss function, optimizer, count the loss and accuracy for each epoches. About the best parameters, I set 3 conditions to decide if I have to save the models: Get the highest accuracy score, Get the lowest loss score, and save all the models after the epoch num that I set. By these methods, I trained three models which each apply different methods.

1. Using ResNext50 [8, 9], training 256, using RandAugment.
2. Using ResNext50 [8, 9], training 288, using TrivialAugmentWide.
3. Using ResNet50 [8, 10], training 224, using RandAugment

After training one time, I searched for a fine-tuning method [11]. By loading my model and modifying the epoches time and the learning-rate, I approved the first and the second models(which means their val_acc increased and val_loss decreased).

About the test.py, I decided to merge the score of the 3 models and take the average score. First, I resized the images into 224, 256, 288 three sizes. Then I prepared the flipped images for each image. Therefore, there were 6 scores in total. Finally each test image was graded by the mean value of these 6 scores. These are all methods that I apply in this homework.

III. Additional experiments

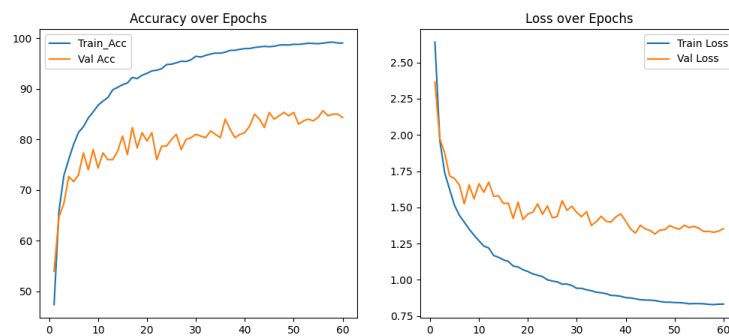
I noticed I don't need to only take the average score of 3 models. Therefore, I try to add the weight on each model. first I set (0.5, 0.3, 0.2), however it wasn't effective. And I failed after several tries. I think this is not a bad idea. It is because in this homework we all use the Resnet as the backbone. Their generalization abilities are too low so that it has no effect. Also, I only apply three models. I believe that if we want to apply multiple models and ensemble method, we should import more models. But these may be difficult according to the parameters limitation.

IV. Results

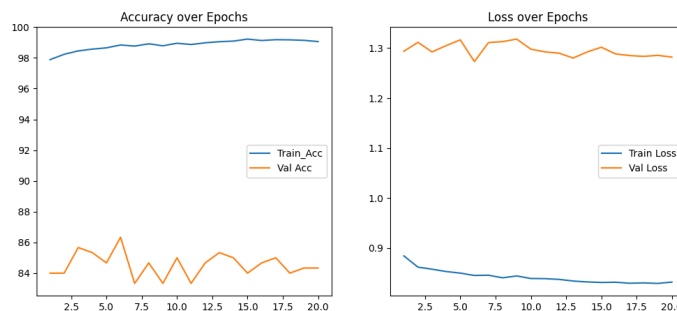
These are my experiments results plots and the scoreboard that I take in the codabench:

ResNext, trivial

first time training:

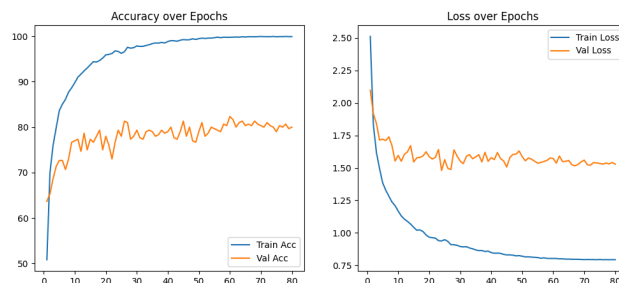


second time tuning:

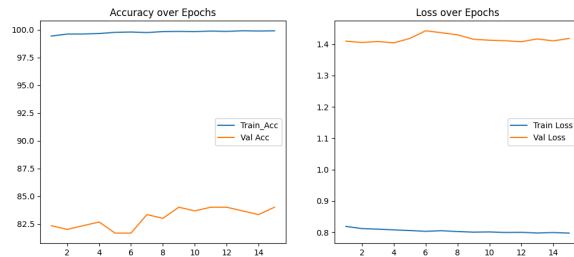


ResNext, RandAugment:

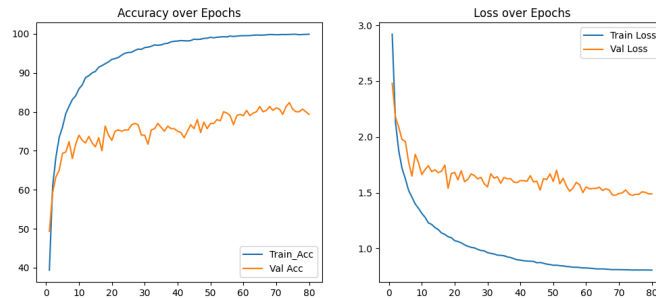
first time training:



second time tuning:



ResNet:



Observing the graph, I noticed that a classic overfitting curve was not observed. However, I stopped the training process because the training accuracy was approaching 100%. I am uncertain whether the absence of this typical pattern is simply due to stopping early, or if there are other underlying reasons why the accuracy and loss curves did not exhibit the expected behavior.

Scoreboard:

ID # ▾	File name	Date	Status	Score	Actions
657784	solution7.zip	2026-03-31 00:35	Finished	0.90	🏠 ⬅️ 🗑️
657776	solution6.zip	2026-03-31 00:32	Finished	0.91	🏠 ⬅️ 🗑️
657341	solution5_2.zip	2026-03-30 20:14	Finished	0.89	🏠 ⬅️ 🗑️
657330	solution5.zip	2026-03-30 20:06	Finished	0.90	🏠 ⬅️ 🗑️
656790	solution4.zip	2026-03-30 11:15	Finished	0.92	✅ ⬅️
656421	solution3.zip	2026-03-30 02:00	Finished	0.92	🏠 ⬅️ 🗑️
656024	solution2.zip	2026-03-29 21:46	Finished	0.89	🏠 ⬅️ 🗑️
655757	solution1_v2.zip	2026-03-29 17:58	Finished	0.67	Delete Submission
655604	solution1.zip	2026-03-29 15:40	Finished	0.08	
655050	solution2.zip	2026-03-29 05:04	Finished		🏠 ⬅️ 🗑️
655046	solution2.zip	2026-03-29 05:02	Failed ❓		🗑️
655037	solution1.zip	2026-03-29 04:58	Failed ❓		🗑️

About the scoreboard, I am reflecting on whether I should have explored more academic methods that require specialized implementations, rather than simply relying on the standard techniques commonly used by general PyTorch users.

V. Reference

- [1] PyTorch Contributors: Transforming and augmenting images. PyTorch/Torchvision documentation, <https://pytorch.org/vision/stable/transforms.html>. Last accessed 31 Mar 2026.
- [2] Deng, J., Dong, W., Socher, R., Li, L.J., Li, K., Fei-Fei, L.: ImageNet: A large-scale hierarchical image database. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 248--255 (2009).
- [3] Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. In: International Conference on Learning Representations (ICLR) (2015).
- [4] PyTorch Contributors: CrossEntropyLoss. PyTorch documentation, <https://pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html>. Last accessed 31 Mar 2026.
- [5] Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019).
- [6] PyTorch Contributors: How to adjust learning rate. PyTorch documentation, <https://pytorch.org/docs/stable/optim.html#how-to-adjust-learning-rate>. Last accessed 31 Mar 2026.
- [7] Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Borisov, E., Warden, P., Liao, C., Wu, H.: Mixed precision training. In: International Conference on Learning Representations (ICLR) (2018).
- [8] PyTorch Contributors: Models and pre-trained weights. PyTorch/Torchvision documentation, <https://pytorch.org/vision/stable/models.html>. Last accessed 31 Mar 2026.
- [9] Xie, S., Girshick, R., Dollár, P., Tu, Z., He, K.: Aggregated residual transformations for deep neural networks. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 1492--1500 (2017).
- [10] He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR). pp. 770--778 (2016).
- [11] Yosinski, J., Clune, J., Bengio, Y., Lipson, H.: How transferable are features in deep neural networks? In: Advances in Neural Information Processing Systems (NeurIPS). pp. 3320--3328 (2014).